



TROUBLESHOOTING HANDBOOK

VIRTUAL CONTROL
VMWARE CLOUD FOUNDATION SOLUTIONS

VCF Cloud Builder Troubleshooting Handbook

Field-tested troubleshooting handbook for Cloud Builder bring-up against an offline depot, covering HTTPS server, firewall, Java trust, and verification.

CLOUD BUILDER

TROUBLESHOOTING

BRING-UP

DIAGNOSTICS

VCF 9.0

VMware Cloud Foundation

VMware Cloud Foundation 9.0.1 -- Cloud Builder Offline Depot Troubleshooting Handbook

Document Information

Field	Value
VCF Version	9.0.1.0
Cloud Builder	VCF 9.0.1
Depot Server OS	Windows (Python HTTPS server)
Date	January 2026
Classification	Internal / Lab Reference

Table of Contents

- [1. Architecture Overview](#)
 - [2. HTTPS Server Configuration](#) - 2.1 [Problem: Secure Protocol Communication Error](#) - 2.2 [Solution: Python HTTPS Server](#) - 2.3 [SSL Certificate Generation](#)
 - [3. Network and Firewall Configuration](#) - 3.1 [Problem: Connection Timed Out](#) - 3.2 [Root Cause: Windows Firewall Public Profile Ignores Local Rules](#) - 3.3 [Solution](#) - 3.4 [Verification](#)
 - [4. Java Certificate Trust](#) - 4.1 [Problem: Secure Protocol Communication Error \(After Network Fix\)](#) - 4.2 [Root Cause](#) - 4.3 [Solution: Import Certificate into Cloud Builder Java Truststore](#) - 4.4 [Certificate Regeneration](#)
 - [5. Offline Depot Directory Structure](#) - 5.1 [Problem: Failed to Validate or Retrieve Component Files](#) - 5.2 [Root Cause](#) - 5.3 [Correct Directory Structure](#) - 5.4 [Key Lesson](#) - 5.5 [Setup Procedure](#)
 - [6. Nested ESXi Host Preparation](#) - 6.1 [Problem: Zero SSD Devices for vSAN Cache Tier](#) - 6.2 [Root Cause](#) - 6.3 [Solution: Mark Virtual Disks as SSD](#) - 6.4 [Problem: Stale vSAN Disk Group Claims](#) - 6.5 [Solution: Remove Stale vSAN Claims](#)
 - [7. ESXi Network Configuration](#) - 7.1 [Problem: Too Many vSphere Standard Switches](#) - 7.2 [Root Cause](#) - 7.3 [Solution](#)
 - [8. Cloud Builder Service Reference](#) - 8.1 [Key Services](#) - 8.2 [Service Commands](#) - 8.3 [502 Errors After Service Restart](#)
 - [9. Quick Reference Checklist](#)
 - [10. References](#)
-

1. Architecture Overview

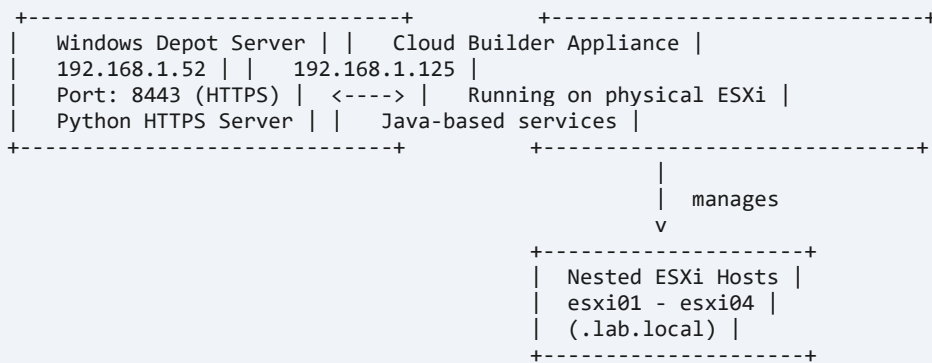
Where commands run in this handbook. Cloud Builder troubleshooting touches:

- **Windows workstation host (PowerShell / CMD)** — Python HTTPS depot server, OpenSSL cert generation, Test-NetConnection for reachability.
- **Cloud Builder appliance shell (SSH)** — appliance-only operations: keytool for cert import, systemctl for service restart, log inspection under /var/log/vmware/vcf/.
- **Cloud Builder UI (browser)** — https://<cloud-builder-fqdn>/ for deployment workflow, depot configuration form, log download.

Each section header notes the environment in parentheses where ambiguous. Replace any credential placeholder like <your-password> with values from your vault.

This handbook documents the end-to-end troubleshooting process for configuring VMware Cloud Foundation 9.0.1 Cloud Builder to consume an offline software depot served from a Windows machine over HTTPS.

Environment Layout



Depot Server (192.168.1.52): A Windows workstation running a custom Python HTTPS server on port 8443. This server hosts the full VCF 9.0.1 offline depot, including all component binaries, metadata manifests, and vSAN HCL data.

Cloud Builder (192.168.1.125): The VCF Cloud Builder appliance, deployed on a physical ESXi host. Cloud Builder orchestrates the bringup workflow, validating the depot, checking ESXi host readiness, and deploying vCenter, NSX, SDDC Manager, and associated components.

Nested ESXi Hosts (esxi01--esxi04): Four nested ESXi 9.0.1 virtual machines running inside VMware Workstation, serving as the management domain compute hosts. These hosts require specific VMX-level configuration to function correctly with vSAN in a nested environment.

2. HTTPS Server Configuration

2.1 Problem: Secure Protocol Communication Error

Symptom: Cloud Builder displays the following error during depot validation:

Secure protocol communication error, check logs for more details

Root Cause: Python's built-in `http.server` module, when wrapped with a basic `ssl.wrap_socket()` call, triggers TLS renegotiation during request handling. Java's SSL/TLS implementation (used by Cloud Builder's backend services) rejects mid-stream renegotiation by default, causing the connection to fail at the application layer even though the initial TLS handshake succeeds.

2.2 Solution: Python HTTPS Server

The following server script resolves all TLS compatibility issues with Cloud Builder's Java runtime. It has been tested and confirmed working with VCF 9.0.1 Cloud Builder.

File: `https_server.py`

```
#!/usr/bin/env python3
"""
HTTPS server for VCF Offline Depot
Serves files with TLS 1.2+ for Cloud Builder compatibility
"""

import http.server
import ssl
import os
import sys
import base64
import socketserver
from functools import partial

# Configuration
PORT = 8443
CERT_FILE = 'server.crt'
KEY_FILE = 'server.key'
USERNAME = 'admin'
PASSWORD = 'admin'

class AuthHandler(http.server.SimpleHTTPRequestHandler):
    protocol_version = "HTTP/1.1"

    def __init__(self, *args, directory=None, **kwargs):
        super().__init__(*args, directory=directory, **kwargs)

    def do_HEAD(self):
        if not self.authenticate():
            return
        super().do_HEAD()

    def do_GET(self):
        if not self.authenticate():
            return
        super().do_GET()

    def authenticate(self):
        auth_header = self.headers.get('Authorization')
        if auth_header is None:
            self.send_auth_request()
            return False

        try:
            auth_type, credentials = auth_header.split(' ', 1)
            if auth_type.lower() != 'basic':
                self.send_auth_request()
                return False

            decoded = base64.b64decode(credentials).decode('utf-8')
            username, password = decoded.split(':', 1)

            if username == USERNAME and password == PASSWORD:
```

```

        return True
    except Exception:
        pass

    self.send_auth_request()
    return False

def send_auth_request(self):
    self.send_response(401)
    self.send_header('WWW-Authenticate', 'Basic realm="VCF Depot"')
    self.send_header('Content-type', 'text/html')
    self.send_header('Content-Length', '23')
    self.send_header('Connection', 'close')
    self.end_headers()
    self.wfile.write(b'Authentication required')

def log_message(self, format, *args):
    msg = f"{self.client_address[0]} - {format % args}"
    print(msg, flush=True)
    with open('depot_access.log', 'a') as f:
        f.write(msg + '\n')

class ThreadedHTTPServer(socketserver.ThreadingMixIn, http.server.HTTPServer):
    daemon_threads = True

def run_server():
    os.chdir(os.path.dirname(os.path.abspath(__file__)))

    # Create SSL context
    context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    context.minimum_version = ssl.TLSVersion.TLSv1_2
    context.maximum_version = ssl.TLSVersion.TLSv1_3

    # Disable post-handshake renegotiation (Java clients reject it)
    if hasattr(context, 'post_handshake_auth'):
        context.post_handshake_auth = False

    # Disable session tickets to avoid renegotiation issues
    context.options |= ssl.OP_NO_TICKET
    context.options |= getattr(ssl, 'OP_NO_RENEGOTIATION', 0)

    # Load certificate
    context.load_cert_chain(CERT_FILE, KEY_FILE)

    # Broad cipher support for Java/VMware compatibility
    try:
        context.set_ciphers('DEFAULT:!aNULL:!MD5:!DSS')
    except ssl.SSLError:
        pass

    handler = partial(AuthHandler, directory=os.getcwd())
    server = ThreadedHTTPServer(('0.0.0.0', PORT), handler)
    server.socket = context.wrap_socket(server.socket, server_side=True)

    print(f"VCF Offline Depot Server")
    print(f"=====")
    print(f"Serving: {os.getcwd()}")
    print(f"URL: https://<IP>:{PORT}/")
    print(f"Credentials: {USERNAME} / {PASSWORD}")
    print(f"TLS: 1.2 - 1.3")
    print(f"Press Ctrl+C to stop")

    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\nStopped.")
        server.shutdown()

```

```
if __name__ == '__main__':
    run_server()
```

Key Design Decisions

Decision	Rationale
<code>protocol_version = "HTTP/1.1"</code>	Java HTTP clients expect HTTP/1.1 persistent connections. HTTP/1.0 causes connection management issues.
<code>OP_NO_RENEGOTIATION</code>	Prevents OpenSSL from initiating TLS renegotiation, which Java's SSLEngine rejects by default. This is the primary fix for the "Secure protocol communication error."
<code>OP_NO_TICKET</code>	Disables TLS session tickets, which can trigger implicit renegotiation on session resumption.
<code>ThreadingMixIn</code>	Cloud Builder makes concurrent HTTP requests to validate multiple components simultaneously. A single-threaded server would serialize these and potentially cause timeouts.
<code>Content-Length + Connection: close on 401</code>	Ensures Java's HTTP client correctly handles authentication challenge responses without hanging on incomplete reads.
<code>DEFAULT:!aNULL:!MD5:!DSS cipher suite</code>	Provides broad cipher compatibility to ensure Java's preferred cipher suites are available, while excluding known-insecure options.
File-based access logging	Writes all requests to <code>depot_access.log</code> for post-hoc troubleshooting of which paths Cloud Builder requests.

2.3 SSL Certificate Generation

Generate a 4096-bit self-signed certificate with proper Subject Alternative Name (SAN) entries. Cloud Builder's Java runtime validates SAN fields, so the certificate must include the server's IP address.

```
MSYS_NO_PATHCONV=1 openssl req -x509 -newkey rsa:4096 \
  -keyout server.key -out server.crt \
  -sha256 -days 365 -nodes \
  -subj "/C=US/ST=Lab/L=Lab/O=VCF-Depot/CN=<YOUR_IP>" \
  -addext "subjectAltName=IP:<YOUR_IP>,DNS:<YOUR_IP>,DNS:localhost" \
  -addext "keyUsage=digitalSignature,keyEncipherment" \
  -addext "extendedKeyUsage=serverAuth"
```

Replace `<YOUR_IP>` with the actual IP address of your depot server (e.g., `192.168.1.52`).

Note: The `MSYS_NO_PATHCONV=1` environment variable is required when running this command on Windows with Git Bash. Without it, MSYS will mangle the `-subj` parameter by interpreting the leading slash as a Unix path and converting it to a Windows path (e.g., `/C=US` becomes `C:\=US`).

3. Network and Firewall Configuration

3.1 Problem: Connection Timed Out

Symptom: Cloud Builder reports the following error during depot connectivity check:

Connect to https://192.168.1.52:8443 failed: Connection timed out

The HTTPS server console shows zero incoming requests despite being confirmed as listening on `0.0.0.0:8443`.

3.2 Root Cause: Windows Firewall Public Profile Ignores Local Rules

The network adapter on the Windows depot server was classified under the **Public** firewall profile. Under this profile, Windows applies a restrictive policy:

```
LocalFirewallRules: N/A (GPO-store only)
```

This means that **all locally created firewall rules are silently ignored**. Even though a rule existed to allow inbound TCP 8443, it had no effect whatsoever. The connection was being dropped at the Windows Firewall level before reaching the Python server.

This is one of the most deceptive Windows networking issues because:

- `netstat` confirms the server is listening.
- `netsh advfirewall firewall show rule` confirms the rule exists.
- Local connections (`curl` from localhost) succeed.
- Yet remote connections silently time out with no log entry.

3.3 Solution

Step 1 -- Change the network profile from Public to Private (elevated PowerShell):

```
Set-NetConnectionProfile -InterfaceAlias "Ethernet 2" -NetworkCategory Private
```

Replace `"Ethernet 2"` with the name of your network adapter. To find the correct name:

```
Get-NetConnectionProfile
```

Step 2 -- Create a firewall rule (if one does not already exist):

```
netsh advfirewall firewall add rule name="VCF Depot HTTPS" dir=in action=allow protocol=tcp  
localport=8443
```

3.4 Verification

From the Windows depot server:

```
curl -k -v https://192.168.1.52:8443/
```

From the Cloud Builder appliance (via SSH):

```
curl -k https://192.168.1.52:8443/
```

Expected result: An HTTP 401 response with body `Authentication required`. This confirms that the network path is open and the HTTPS server is responding correctly.

4. Java Certificate Trust

4.1 Problem: Secure Protocol Communication Error (After Network Fix)

Symptom: After resolving the network/firewall issue:

- `curl -k` from the Cloud Builder appliance successfully reaches the depot server and receives valid responses.
- Cloud Builder's Java application still fails with: "Secure protocol communication error, check logs for more details."
- The depot server access log may show SSL handshake errors, or no incoming requests at all from Cloud Builder's Java processes.

4.2 Root Cause

Cloud Builder's backend services run on Java (OpenJDK 17). Java maintains its own certificate truststore (`cacerts`), which is completely independent of the operating system's certificate store. When `curl` is invoked with the `-k` flag, it bypasses certificate validation entirely. However, Java's `SSLContext` enforces full certificate chain validation by default.

Because the depot server uses a self-signed certificate, Java's TLS handshake fails during certificate verification -- before any HTTP request is sent.

4.3 Solution: Import Certificate into Cloud Builder Java Truststore

SSH into the Cloud Builder appliance and execute the following steps.

Step 1 -- Download the server certificate:

```
openssl s_client -connect 192.168.1.52:8443 </dev/null 2>/dev/null | openssl x509 > /tmp/depot.crt
```

Step 2 -- Locate the Java truststore:

```
find / -name cacerts -path "*/security/*" 2>/dev/null
```

Typical path on Cloud Builder:

```
/usr/lib/jvm/openjdk-java17-headless.x86_64/lib/security/cacerts
```

Step 3 -- Import the certificate:

```
keytool -importcert -alias vcf-depot -file /tmp/depot.crt \  
-keystore /usr/lib/jvm/openjdk-java17-headless.x86_64/lib/security/cacerts \  
-storepass changeit -noprompt
```

Step 4 -- Restart Cloud Builder services:

```
systemctl restart commonsvc domainmanager lcm operationsmanager
```

Step 5 -- Wait for services to initialize, then verify:


```
systemctl is-active commonsvcs domainmanager lcm operationsmanager
```

All four services should report `active`. Allow 2--3 minutes for full initialization before retrying the depot validation in the Cloud Builder UI.

4.4 Certificate Regeneration

If you regenerate the SSL certificate on the depot server (e.g., after changing the server IP or after the certificate expires), you **must** delete the old certificate from the truststore and import the new one.

```
# Delete the old certificate
keytool -delete -alias vcf-depot \
  -keystore /usr/lib/jvm/openjdk-java17-headless.x86_64/lib/security/cacerts \
  -storepass changeit

# Download the new certificate
openssl s_client -connect 192.168.1.52:8443 </dev/null 2>/dev/null | openssl x509 > /tmp/depot.crt

# Import the new certificate
keytool -importcert -alias vcf-depot -file /tmp/depot.crt \
  -keystore /usr/lib/jvm/openjdk-java17-headless.x86_64/lib/security/cacerts \
  -storepass changeit -noprompt

# Restart services to pick up the new certificate
systemctl restart commonsvcs domainmanager lcm operationsmanager
```

5. Offline Depot Directory Structure

5.1 Problem: Failed to Validate or Retrieve Component Files

Symptom: Cloud Builder reports errors such as:

Failed to validate or retrieve component file for VMware Cloud Foundation Operations version 9.0.1.0.24960351

Failed to validate or retrieve component file for VMware vCenter version 9.0.1.0.24957454

These errors appear even when the binary files are physically present on the depot server and accessible via `curl`.

5.2 Root Cause

Two issues contributed to this failure:

1. **Incorrect directory layout.** Binary files were placed in a flat `PROD/bundle/` directory instead of being organized into component-specific subdirectories under `PROD/COMP/<COMPONENT_NAME>/`.
2. **Missing PATCH binaries.** Cloud Builder validates the existence of both `INSTALL` and `PATCH` artifacts for every component listed in `productVersionCatalog.json`. Even though patch files are not used during initial bringup, their absence causes the validation to fail.

5.3 Correct Directory Structure

The directory structure is defined by the `vcf-9.0.1.0-offline-depot-metadata.zip` file (downloaded from Broadcom), combined with the `productVersionCatalog.json` manifest that maps every component to its exact filenames and relative paths.

```
PROD/
|
+-- COMP/
|   +-- ESX_HOST/
|       +-- VMware-VMvisor-Installer-9.0.1.0.24957456.x86_64.iso
|
|   +-- VCENTER/
|       +-- VMware-VCSA-all-9.0.1.0.24957454.iso [INSTALL]
|       +-- VMware-vCenter-Server-Appliance-9.0.1.0.24957454-updaterepo.zip [PATCH]
|
|   +-- NSX_T_MANAGER/
|       +-- nsx-unified-appliance-9.0.1.0.24952114.ova [INSTALL]
|       +-- VMware-NSX-upgrade-bundle-9.0.1.0.24952111.mub [PATCH]
|       +-- VMware-NSX-upgrade-bundle-9.0.1.0.24952111-pre-check.pub [PATCH]
|
|   +-- SDDC_MANAGER_VCF/
|       +-- Compatibility/
|           +-- VMwareCompatibilityData.json
|       +-- VCF-SDDC-Manager-Appliance-9.0.1.0.24962180.ova [INSTALL]
|       +-- VCF-SDDC-Manager-Appliance-Upgrade-9.0.1.0.24962180.tar [PATCH]
|
|   +-- VCFDT/
|       +-- vcf-download-tool-9.0.1.0.24962179.tar.gz [INSTALL]
|
|   +-- VRSLCM/
|       +-- VCF-OPS-Lifecycle-Manager-Appliance-9.0.1.0.24960371.ova [INSTALL]
|       +-- VCF-OPS-Lifecycle-Manager-9.0.1.0.24960371.patch [PATCH]
|
|   +-- VRLI/
|       +-- Operations-Logs-Appliance-9.0.1.0.24960345.ova [INSTALL]
|       +-- Operations-Logs-Appliance-9.0.1.0.24960345.pak [PATCH]
|
|   +-- VROPS/
|       +-- Operations-Appliance-9.0.1.0.24960351.ova [INSTALL]
|       +-- Operations-Upgrade-9.0.1.0.24960352.pak [PATCH]
|
|   +-- VCF_OPS_CLOUD_PROXY/
|       +-- Operations-Cloud-Proxy-9.0.1.0.24960349.ova [INSTALL]
|
|   +-- VRA/
|       +-- vmosp-vcfa-combined-9.0.1.0.24965341.tar [INSTALL]
|
|   +-- VIDB/
|       +-- vidb-external-9.0.1.0.24941398.tar [INSTALL]
|
|   +-- VRNI/
|       +-- VMware-Cloud-Foundation-Operations-for-Networks-9.0.1.0.24950933-collector.ova [INSTALL]
|       +-- VMware-Cloud-Foundation-Operations-for-Networks-9.0.1.0.24950933-platform.ova [INSTALL]
|       +-- VMware-Cloud-Foundation-Operations-for-Networks-9.0.1.0.24950933-upgrade.bundle [PATCH]
|
|   +-- HCX/
|       +-- hcx-unified-appliance-9.0.1.0.24972592.ova [INSTALL]
|       +-- hcx-unified-appliance-upgrade-bundle-9.0.1.0.24972592-signed.tar.gz [PATCH]
|
|   +-- VRO/
|       +-- 011N_VA-9.0.1.0.24923009.ova [INSTALL]
|       +-- 011N_VA-9.0.1.0.24923009-updaterepo.iso [PATCH]
|
|   +-- DLVM/ (empty -- only needed for Deep Learning VMs)
|   +-- VMTTOOLS/ (empty -- only needed if explicitly required)
|   +-- VSAN_FILE_SERVICES/ (empty -- only needed for vSAN File Services)
|
+-- metadata/
|   +-- manifest/
|       +-- v1/
```

```

| |      +-- vcfManifest.json
+-- productVersionCatalog/
| |      +-- v1/
| |      +-- productVersionCatalog.json
| |      +-- productVersionCatalog.sig
+-- vsan/
| |      +-- hcl/
| |      +-- all.json
| |      +-- lastupdatedtime.json

```

5.4 Key Lesson

Cloud Builder validates **both** INSTALL and PATCH binaries for every component listed in `productVersionCatalog.json`. Missing patch files will cause validation failure even though they are not consumed during initial bringup deployment. There is no way to skip or bypass this validation -- all referenced files must be present and accessible via the HTTPS depot.

5.5 Setup Procedure

1. **Extract the metadata archive.** Unzip `vcf-9.0.1.0-offline-depot-metadata.zip` into the depot root directory. This creates the `PROD/` skeleton including `metadata/`, `vsan/hcl/`, and `COMP/SDDC_MANAGER_VCF/Compatibility/` with their respective JSON files.
2. **Create component subdirectories.** Under `PROD/COMP/`, create a subdirectory for each component (e.g., `ESX_HOST`, `VCENTER`, `NSX_T_MANAGER`, etc.).
3. **Place INSTALL binaries.** Copy each component's install artifact (ISO, OVA, TAR, etc.) into the corresponding `PROD/COMP/<COMPONENT>/` directory.
4. **Place PATCH binaries.** Copy each component's patch artifact into the same component directory. Refer to `productVersionCatalog.json` for the exact expected filenames.
5. **Verify accessibility.** For each file, confirm it returns HTTP 200 when accessed through the HTTPS server:

```
bash curl -k -u admin:admin https://192.168.1.52:8443/PROD/COMP/VCENTER/VMware-VCSA-all-9.0.1.0.24957454.iso -I
```

The response should include `HTTP/1.1 200 OK` with the correct `Content-Length`.

6. Nested ESXi Host Preparation

6.1 Problem: Zero SSD Devices for vSAN Cache Tier

Symptom: Cloud Builder validation reports:

ESX Host esxi01.lab.local found zero SSD devices for SSD cache tier

6.2 Root Cause

Virtual SATA disks created in VMware Workstation are not advertised as solid-state devices by default. The hypervisor presents them as rotational (HDD) storage. vSAN requires at least one SSD for the cache tier and will not proceed

without one.

6.3 Solution: Mark Virtual Disks as SSD

Step 1 -- Power off each nested ESXi VM completely (shutdown, not suspend).

Step 2 -- Edit the `.vmx` **file** and add the following lines to mark the vSAN disks as SSD:

```
sata0:3.virtualSSD = 1
sata0:4.virtualSSD = 1
```

Adjust the SCSI/SATA device identifiers (`sata0:3` , `sata0:4`) to match the controller and unit numbers of your vSAN cache and capacity disks. The first number is the controller ID and the second is the disk unit number.

Step 3 -- Power on the ESXi VM. A full power cycle is required -- VMX parameter changes are not picked up on a guest OS reboot.

Step 4 -- Verify inside ESXi via SSH:

```
esxcli storage core device list | grep -E "Display Name|Is SSD"
```

All vSAN-designated disks should report `Is SSD: true`.

6.4 Problem: Stale vSAN Disk Group Claims

Symptom: Disks correctly show as SSD, but Cloud Builder still reports zero SSD devices for the cache tier. Running `esxcli vsan storage list` reveals disks claimed by a previous vSAN configuration with the flag:

```
Used by this host: false
```

This indicates leftover vSAN metadata from a prior deployment is preventing Cloud Builder from recognizing the disks as available.

6.5 Solution: Remove Stale vSAN Claims

Execute the following commands via SSH on each affected ESXi host:

```
# Unmount the capacity (HDD) disk -- use the -d flag
esxcli vsan storage diskgroup unmount \
  -d t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____040000000000000001

# Unmount the cache (SSD) disk -- use the -s flag
esxcli vsan storage diskgroup unmount \
  -s t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____030000000000000001

# Wipe partition tables to remove vSAN metadata
partedUtil mklabel \
  "/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____030000000000000001" gpt
partedUtil mklabel \
  "/vmfs/devices/disks/t10.ATA_____VMware_Virtual_SATA_Hard_Drive_____040000000000000001" gpt

# Verify disks are clean
esxcli vsan storage list

# Persist configuration across reboots
/sbin/auto-backup.sh
```

Important notes:

Consideration	Detail
Unmount flags	Use <code>-d</code> for capacity/HDD disks and <code>-s</code> for SSD/cache disks. Using the wrong flag will produce an error.
<code>partedUtil mklabel gpt</code>	Recreates a clean GPT partition table, effectively removing all vSAN on-disk metadata. This is the correct approach on ESXi.
<code>dd</code> commands	Standard <code>dd if=/dev/zero</code> commands may fail on ESXi with "Function not implemented." Always use <code>partedUtil mklabel</code> instead.
Read-only errors	If a disk reports "Read-only file system" during the wipe, unmount it first using the <code>esxcli vsan storage diskgroup unmount</code> command before retrying.

7. ESXi Network Configuration

7.1 Problem: Too Many vSphere Standard Switches

Symptom: Cloud Builder validation reports:

ESX Host esxi01.lab.local has 2 vSphere Standard Switches (Expecting 1)

There are more than one VM Kernel Adapters configured

7.2 Root Cause

Leftover network configuration from a previous VCF deployment or manual setup remains on the ESXi host. Cloud Builder expects a clean ESXi installation with only:

- One vSphere Standard Switch: **vSwitch0**
- One VMkernel adapter: **vmk0** (management network)

Any additional vSwitches, port groups, or VMkernel adapters will cause validation to fail.

7.3 Solution

Remove the extraneous network objects via SSH on each affected ESXi host:

```
# Remove the extra VMkernel adapter
esxcli network ip interface remove -i vmk30

# Remove the port group from the extra vSwitch
esxcli network vswitch standard portgroup remove -p VSAN -v vcenter-cl01-vds01

# Remove the uplink from the extra vSwitch
esxcli network vswitch standard uplink remove -u vmnic1 -v vcenter-cl01-vds01

# Delete the extra vSwitch
esxcli network vswitch standard remove -v vcenter-cl01-vds01

# Persist the configuration
/sbin/auto-backup.sh
```

Adjust the interface names (`vmk30`), port group names (`VSAN`), uplink names (`vmnic1`), and vSwitch names (`vcenter-c101-vds01`) to match your environment.

Verify:

```
esxcli network vswitch standard list # Should show only vSwitch0
esxcli network ip interface list # Should show only vmk0
```

Warning: If you power cycle the ESXi VM (e.g., to apply VMX changes such as `virtualSSD`), the network configuration may revert to a previous state. Always run `/sbin/auto-backup.sh` after making changes, and re-verify the configuration after any reboot or power cycle.

8. Cloud Builder Service Reference

8.1 Key Services

Service Name	Description
<code>commonsvcs.service</code>	VMware Cloud Foundation Platform Services -- shared platform infrastructure
<code>domainmanager.service</code>	VMware Cloud Foundation Domain Manager -- orchestrates domain lifecycle
<code>lcm.service</code>	VMware Cloud Foundation Lifecycle Management Service -- handles depot and component management
<code>operationsmanager.service</code>	VMware Cloud Foundation Operations Manager -- monitoring and operational tasks
<code>nginx.service</code>	NGINX HTTP reverse proxy -- frontend for the Cloud Builder web UI

8.2 Service Commands

Check status of all key services:

```
systemctl is-active commonsvcs domainmanager lcm operationsmanager
```

Restart all key services:

```
systemctl restart commonsvcs domainmanager lcm operationsmanager
```

List all running VCF-related services:

```
systemctl list-units --type=service --state=running | grep -i -E "vcf|cloud|tomcat|nginx|lightwave"
```

Check for failed services:

```
systemctl list-units --type=service --state=failed
```

8.3 502 Errors After Service Restart

After restarting Cloud Builder services, the web UI may display errors such as:

Http failure response for https://192.168.1.125/... : 502 OK

This is expected behavior. The NGINX reverse proxy starts immediately, but the backend Java services require 2--3 minutes to fully initialize. During this window, NGINX returns 502 (Bad Gateway) because the upstream services are not yet ready.

Resolution: Wait 2--3 minutes and refresh the browser. No action is required.

9. Quick Reference Checklist

Use this checklist before initiating Cloud Builder bringup validation.

Depot Server (Windows)

- [] HTTPS server running and accessible on the designated port (e.g., 8443)
- [] SSL certificate generated with correct SAN entries matching the server IP
- [] Windows network profile set to **Private** (not Public)
- [] Windows Firewall rule allows inbound TCP on the depot port
- [] Verify with `curl -k https://<DEPOT_IP>:8443/` from a remote machine

Depot Content

- [] `PROD/metadata/manifest/v1/vcfManifest.json` exists and is accessible
- [] `PROD/metadata/productVersionCatalog/v1/productVersionCatalog.json` exists and is accessible
- [] `PROD/metadata/productVersionCatalog/v1/productVersionCatalog.sig` exists and is accessible
- [] `PROD/vsan/hcl/all.json` exists and is accessible
- [] `PROD/vsan/hcl/lastupdatedtime.json` exists and is accessible
- [] `PROD/COMP/SDDC_MANAGER_VCF/Compatibility/VmwareCompatibilityData.json` exists and is accessible
- [] All component **INSTALL** binaries placed in `PROD/COMP/<COMPONENT>/`
- [] All component **PATCH** binaries placed in `PROD/COMP/<COMPONENT>/`

Cloud Builder Appliance

- [] Depot server certificate imported into Java truststore (`cacerts`)
- [] Cloud Builder services restarted after certificate import
- [] All four services report `active` (`commonsvcs` , `domainmanager` , `lcm` , `operationsmanager`)

Nested ESXi Hosts

- [] Each host has exactly **one** vSphere Standard Switch (vSwitch0)
- [] Each host has exactly **one** VMkernel adapter (vmk0 -- management)
- [] vSAN disks marked as SSD in VMX configuration (`virtualSSD = 1`)
- [] No stale vSAN disk group claims (`esxcli vsan storage list` returns empty)
- [] ESXi configuration persisted (`/sbin/auto-backup.sh` executed after all changes)

10. References

- [Set Up an Offline Depot Web Server for VMware Cloud Foundation](#)
 - [Download Install Binaries to an Offline Depot](#)
 - [VCF 9.x Bringup Series -- Downloading Binaries](#)
 - [VCF Software Depot Structure Deep Dive](#)
-

This document was compiled from hands-on troubleshooting of a VCF 9.0.1 Cloud Builder deployment with an offline depot in a nested lab environment. Last updated January 2026.